Document number: P3638R0
Date: 2025-02-14
Project: Programming Language C++
Reference: ISO/IEC 14882:2024
Reply to: Jens Maurer
jens.maurer@gmx.net

Core Language Working Group "ready" Issues for the February, 2025 meeting

References in this document reflect the section and paragraph numbering of document WG21 N5001.

2549. Implicitly moving the operand of a *throw-expression* in unevaluated contexts

Section: 7.5.5.3 [expr.prim.id.qual] **Status:** ready **Submitter:** Richard Smith **Date:** 2022-03-11

Consider:

```
void f() {
  X x;
  // Is x an lvalue or an xvalue here?
  void g(int n = (decltype((throw x, 0))()));  // status quo: x is move-eligible here
}

void f() {
  X x;
  struct A {
    void g() {
      try {
        struct Y {
          // Is x an lvalue or an xvalue here?
          void h(int n = (decltype((throw x, 0))()));
        };
      } catch (...) { }
    }
  };
}
```

11.9.6 [class.copy.elision] paragraph 3 specifies:

> An *implicitly movable entity* is a variable of automatic storage duration that is either a non-volatile object or an rvalue reference to a non-volatile object type. In the following copy-initialization contexts, a move operation is first considered before attempting a copy operation:
>
> - ...
> - if the operand of a *throw-expression* (7.6.18 [expr.throw]) is a (possibly parenthesized) *id-expression* that names an implicitly movable entity that belongs to a scope that does not contain the *compound-statement* of the innermost *try-block* or *function-try-block* (if any) whose *compound-statement* or *ctor-initializer* contains the *throw-expression*,

Thus, in the first example above, `x` is treated as an xvalue, but it is treated as an lvalue in the second example. This outcome is surprising.

(P2266R2 (Simpler implicit move) moved this wording, introduced by P1825R0 (Merged wording for P0527R1 and P1155R3), from 11.9.6 [class.copy.elision] to 7.5.5.2 [expr.prim.id.unqual].)

Proposed resolution [SUPERSEDED]:

Change in 7.5.5.2 [expr.prim.id.unqual] paragraph 4:

> An *implicitly movable entity* is a variable ~~of~~ **with** automatic storage duration that is either a non-volatile object or an rvalue reference to a non-volatile object type. ~~In the following contexts, an~~ **An** *id-expression* is *move-eligible*~~:~~ **if**
>
> - **it names an implicitly movable entity declared in the body or *parameter-declaration-clause* of the innermost enclosing function or *lambda-expression* and**
> - ~~If~~ the ~~*id-expression*~~ (possibly parenthesized) ***id-expression*** is the operand of
>   - a `return` or `co_return` statement~~, and names an implicitly movable entity declared in the body or *parameter-declaration-clause* of the innermost enclosing function or *lambda-expression*~~ or
>   - ~~if the *id-expression* (possibly parenthesized) is the operand of~~ a **potentially-evaluated** *throw-expression*, ~~and names an implicitly movable entity that belongs to a scope that does not contain the *compound-statement* of the innermost *lambda-expression*, *try-block*, or *function-try-block* (if any) whose *compound-statement* or *ctor-initializer* encloses the *throw-expression*~~ **where no *try-block* or *function-try-block* intervenes between the declaration of the entity and the innermost enclosing scope of the *throw-expression***.

Additional notes (December, 2024)

Treating potentially-evaluated expressions differently (as opposed to unevaluated ones) is surprising.

Proposed resolution (approved by CWG 2025-02-14):

Change in 7.5.5.2 [expr.prim.id.unqual] paragraph 4:

> An *implicitly movable entity* is a variable ~~of~~ **with** automatic storage duration that is either a non-volatile object or an rvalue reference to a non-volatile object type. ~~In the following contexts, an~~ **An** *id-expression* is *move-eligible*~~:~~ **if**
>
> - it names an implicitly movable entity,
> - ~~If~~**it is** the ~~*id-expression*~~ (possibly parenthesized) ~~is the~~ operand of a `return` or `co_return` statement~~, and names an implicitly movable entity declared in the body or *parameter-declaration-clause* of the innermost enclosing function or *lambda-expression*~~ or ~~if the *id-expression* (possibly parenthesized) is the operand~~ of a *throw-expression*, and ~~names an implicitly movable entity that belongs to a scope that does not contain the *compound-statement* of the innermost *lambda-expression*, *try-block*, or *function-try-block* (if any) whose *compound-statement* or *ctor-initializer* encloses the *throw-expression*~~
> - each intervening scope between the declaration of the entity and the innermost enclosing scope of the *id-expression* is a block scope and, for a *throw-expression*, is not the block scope of a *try-block* or *function-try-block*.

2703. Three-way comparison requiring strong ordering for floating-point types, take 2

Section: 11.10.3 [class.spaceship] **Status:** ready **Submitter:** Richard Smith **Date:** 2023-02-13

The resolution accepted for issue 2539 does not actually address the example in the issue, because overload resolution is never performed for expressions involving only built-in types.

Proposed resolution (2025-01-13, approved by CWG 2025-02-14):

Change in 11.10.3 [class.spaceship] paragraph 1 as follows:

> The synthesized three-way comparison of type R (17.11.2 [cmp.categories]) of glvalues a and b of the same type is defined as follows:
>
> - If a <=> b is usable (11.10.1 [class.compare.default]) and can be explicitly converted to R using static_cast, `static_cast<R>(a <=> b)`.
> - Otherwise, if **`a <=> b` is usable or** overload resolution for `a <=> b` is performed and finds at least one viable candidate, the synthesized three-way comparison is not defined.
> - Otherwise, ...

2943. Discarding a void return value

Section: 9.12.10 [dcl.attr.nodiscard] **Status:** ready **Submitter:** Brian Bi **Date:** 2024-10-24

(From submission #628.)

A warning is currently, but ought not to be, encouraged for a call to a [[nodiscard]] function with a void return type. Such a situation may arise for dependent return types. For example:

```
[[nodiscard]] void f();
template<class T> [[nodiscard]] T g();

void h() {
  f();              // suggested change: warning no longer recommended
  (void)f();        // warning not recommended
  g<int>();         // warning recommended
  g<void>();        // suggested change: warning no longer recommended
  (void)g<void>();  // warning not recommended
}
```

Proposed resolution (2025-01-21, approved by CWG 2025-02-14):

Change in 9.12.10 [dcl.attr.nodiscard] paragraph 4 as follows:

> Recommended practice: Appearance of a nodiscard call as a potentially-evaluated discarded-value expression (7.2 [expr.prop]) **of non-void type** is discouraged unless explicitly cast to void. Implementations should issue a warning in such cases. The value of a *has-attribute-expression* for the nodiscard attribute should be 0 unless the implementation can issue such warnings.

2970. Races with `volatile sig_atomic_t` bit-fields

Section: 6.9.2.2 [intro.races] **Status:** ready **Submitter:** Hubert Tong **Date:** 2024-12-17

(From submission #654.)

Subclause 6.9.2.2 [intro.races] paragraph 22 specifies:

> Two accesses to the same object of type `volatile std::sig_atomic_t` do not result in a data race if both occur in the same thread, even if one or more occurs in a signal handler. ...

This provision applies to bit-fields as well, because bit-fields are objects (6.8.1 [basic.types.general] paragraph 4). However, in practice bit-fields are not updated atomically and are subject to tearing.

Proposed resolution (2025-01-20, approved by CWG 2025-02-14):

Change in 6.9.2.2 [intro.races] paragraph 22 as follows:

> Two accesses to the same **non-bit-field** object of type `volatile std::sig_atomic_t` do not result in a data race if both occur in the same thread, even if one or more occurs in a signal handler. ...

2990. Exporting redeclarations of namespaces

Section: 10.2 [module.interface] **Status:** ready **Submitter:** EWG/CWG **Date:** 2025-01-10

Issue 2921 fixed the following issue with namespaces:

```
export module M;
namespace N { // external linkage, attached to global module, not exported
  void f();
}
namespace N { // error: exported namespace, redeclares non-exported namespace
  export void g();
}
```

This is considered a CWG consistency / wording fix. However, the change for that issue also allowed:

```
module;
#include "header"
export module wrap;

export using ::feature;               // already allowed previously
export extern "C++" void feature();   // newly allowed by CWG2921
```

The CWG chair had neglected to run this new feature past EWG prior to plenary-approving issue 2921 in Wroclaw. Subsequent discussion on the EWG reflector surfaced concerns about missing syntactic differentiation between an intended export-by-redeclaration and an accidental declaration of a different entity because of a slight signature mismatch.

This issues seeks to limit the change to namespaces only; any additional feature in this area should be presented to EWG via a paper.

Proposed resolution (approved by CWG 2025-02-14):

Change in 10.2 [module.interface] paragraph 6 as follows:

> A redeclaration of an entity X is implicitly exported if X was introduced by an exported declaration; otherwise it shall not be exported ~~if it is attached to a named module~~ **unless it is a namespace**. [Example:
>
> ```
> export module M;
> struct S { int n; };
> typedef S S;
> export typedef S S;      // OK, does not redeclare an entity
> export struct S;         // error: exported declaration follows non-exported declaration
> namespace N {            // external linkage, attached to global module, not exported
>   void f();
> }
> namespace N {            // OK, exported namespace redeclaring non-exported namespace
>   export void g();
> }
> ```
>
> -- end example]